道路垃圾视觉识别与分类系统

摘要

随着校园、园区及城市公共道路人流量与消费活动不断增加，塑料瓶、易拉罐、纸张、塑料袋等生活垃圾被随意丢弃的现象日益突出，影响道路环境整洁与校园形象的同时也增加了环卫工人的巡检负担。传统人工巡检方式存在效率低、主观性强、分类统计困难等不足，难以满足精细化、智能化的环卫管理需求，亟需引入"深度学习算法+边缘异构计算"相融合的新技术方案。

本项目基于 Xilinx Zynq UltraScale+ MPSoC 的 KV260 平台，构建了一套道路垃圾识别系统。系统通过双目摄像头采集校园、房间及公共道路等场景图像，针对金属罐（Cans）、玻璃瓶（Glass_Bottles）、纸类（Paper）、塑料（Plastic）、纺织物（Textile）及其他垃圾（other）六类常见垃圾，采用 YOLOv5s 深度学习模型，基于 PyTorch 框架对垃圾数据集进行训练，实现多类别垃圾识别，并对垃圾类别、数量、位置和置信度进行分析[5]。

整个系统方案围绕软硬件分工协作的异构架构展开。PS 侧主要负责系统管理，运行嵌入式 Linux 完成进程调度与资源分配；PL 侧则借助硬件加速，把图像预处理、AI 推理和结果处理串联为一条完整通路。模型经 Vitis AI 工具链量化编译后部署于 DPU，使嵌入式端得以兼顾较高识别精度与实时定位能力。检测输出的垃圾类别、目标距离、数量及系统运行状态，可由电脑界面或串口屏实时呈现，并进一步生成控制信号驱动激光振镜，依次标记各个目标位置[6][7]。

面对校园、房间及公共道路等复杂环境，系统仍能较准确地识别并标记垃圾目标，这正是边缘智能与硬件加速结合后所呈现的实际效果，可为环卫巡检、垃圾清理、分类投放和后续无人清扫设备提供辅助信息。结合当前开发进度，后续工作可向多目标跟踪、清扫路径规划、机械抓取以及垃圾自动分类等方向延伸，为智慧校园和城市道路环卫智能化积累更多技术经验。

第一部分　作品概述

1.1 功能与特性

本作品面向校园、房间及公共道路的垃圾巡检需求，把图像采集、智能识别、距离测量和激光标记整合在同一嵌入式平台上，主要功能如下：

- **多类别识别**：区分金属罐、玻璃瓶、纸类、塑料、纺织物和其他六类垃圾，并给出各目标的数量、位置与置信度。
- **双目测距**：利用左右两路图像的视差估算目标距离，进而得到其空间位置。
- **激光标记**：依据目标坐标控制激光振镜，逐个指向并标记垃圾。
- **信息显示**：类别、数量、距离及运行状态可在显示器 UI 界面实时查看。

1.2 应用领域

- **校园与公共环卫**：可用于教室、宿舍、校内道路等区域的垃圾巡查，自动完成六类垃圾的检测、定位与计数，并用激光指示位置，比人工逐处排查更省人力，也为后续清理和分类投放提供依据。
- **工业生产**：同样的视觉检测加激光指示思路，可用于生产线上的异物排查、表面标记和物料分拣，适合对节拍有要求的电子制造、物流分拣等环节。
- **农业环境**：可拓展到农田、果园和温室，识别农药瓶、地膜碎片、塑料袋、纸箱等农业废弃物，结合测距与激光标记，辅助田间清理和废弃物回收。
- **城市环境管理**：在城市道路、公园等场所，可实现辅助乃至无人化的垃圾发现与标记，减少人工巡视，提高环境维护的自动化水平。

1.3 主要技术特点

FPGA 具有并行度高、延迟低、可按需定制的特点，适合做边缘端的实时数据处理。本作品以 Kria KV260 为平台，配合 Vivado、Vitis 与 Vitis AI 工具链，主要技术点如下：

- **图像采集与测距**：双目摄像头获取场景图像，由视差估算目标距离。
- 模型训练与压缩：在 PyTorch 中用六类垃圾数据集训练 YOLOv5s，再经 Vitis AI 量化为 INT8。
- **硬件推理加速**：量化模型运行在 PL 侧 DPU 上，由硬件承担网络推理。

• **软硬件协同**：PS 端 Linux 负责调度与结果解析，PL 端负责预处理与加速运算。

• **激光指向控制**：DAC 输出模拟电压驱动激光振镜，实现快速准确的指向。

• **UI 界面设计**：基于 Linux 系统的火狐浏览器，设计美观人性化的交互窗口，观测状态实时显示，配合按键和进度条，方便用户了解清扫情况并操作激光。

1.4 主要性能指标

如下表所示。

表 1 性能指标

序号	指标	内容 / 结果
1	识别准确度	验证集 PR 曲线 mAP@0.5 为 0.980
2	推理速度	DPU 部署后的平均帧率：约 166FPS
3	定位误差	X、Y 方向的均方根误差分别约为 4.10 mm、5.18 mm
4	振镜精度	指向误差：2~3mm(距离约 80cm 时)

1.5 主要创新点

（1）算法与硬件结合：将 YOLOv5s 的激活函数改为 ReLU，量化后部署到 DPU，兼顾精度与速度。

（2）板上全流程一体化：识别结果在板内直接换算为振镜控制量，不依赖上位机。

（3）信息融合：类别、数量、距离和置信度同时输出，便于监控与决策。

（4）便于扩展：基于 Vivado 与 Vitis 开发，模型和硬件可分别升级，后续可加入跟踪、路径规划等功能。

（5）交互体验：UI 界面功能齐全，画面美观。

1.6 设计流程

我们的设计按“硬件平台—算法模型—应用软件”三步推进。先在 Vivado 中搭建工程，集成 Zynq PS、DPU、摄像头接口、测距模块和 DAC 控制模块；再用 PyTorch 训练 YOLOv5s 得到 .pt 权重，由 Vitis AI 量化并编译为 DPU 可执行的 .xmodel；最后在 Vitis 中编写 PS 端程序，通过 AXI 总线调用 PL 加速模块并联动振镜，把识别、测距、显示和标记连成一体。

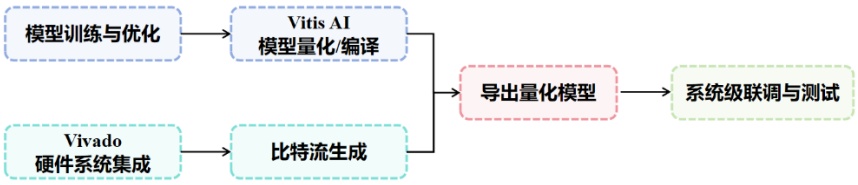


图 1 设计流程

第二部分 系统组成及功能说明

2.1 整体介绍

系统由数据处理与硬件控制两条链路构成（图 2）。双目摄像头采集场景图像后送入 KV260，板上依次完成预处理、YOLOv5s 推理和后处理[3]，得到各目标的类别、置信度与像素坐标；同时利用左右图像的视差计算目标距离，还原其空间位置。随后 PS 侧程序把目标坐标换算为振镜的偏转量，经 SPI 下发给 DAC，DAC 输出的模拟电压驱动激光振镜转到指定角度，激光点便落在目标上。识别结果与运行状态则并行送往电脑界面或串口屏显示。

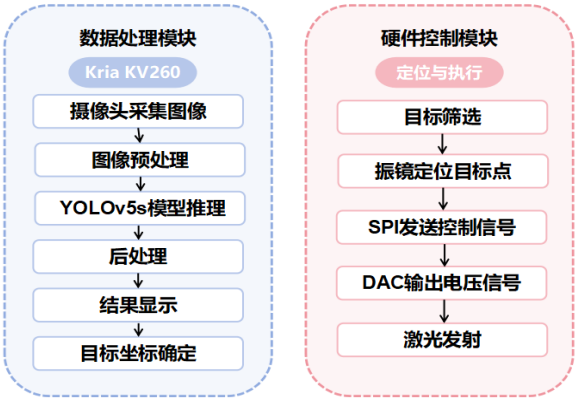


图 2 系统整体结构图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件以 Kria KV260 为中心，外接双目摄像头、DAC 模块、激光振镜和供电电源。KV260 同时承担推理与决策：接收图像、运行 YOLOv5s，并把结果换算为控制量输出给 DAC；DAC 与振镜构成执行端，完成激光指向[1]。

2.2.2 机械设计介绍

以下是为搭建装置设计的模块支架，采用 SolidWorks 建模：

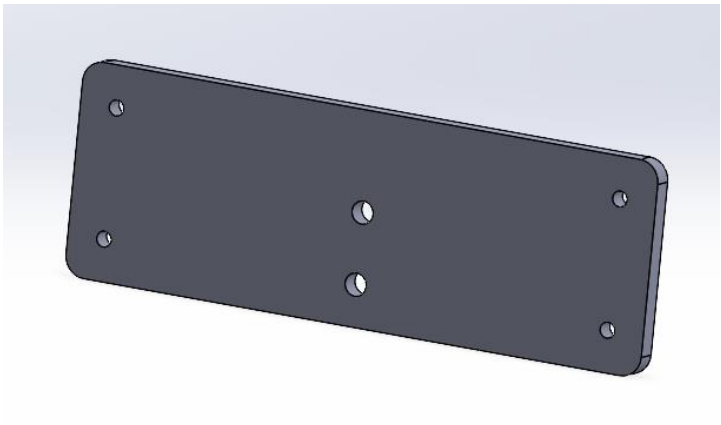


图 3 双目摄像头支架

由于激光振镜平放时受角度限制，难以打到较近的地面上，因此为其设计 36°斜坡支架。

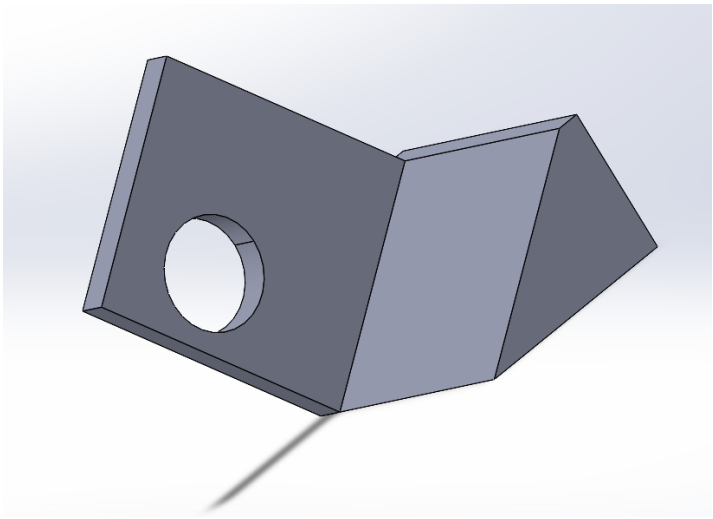
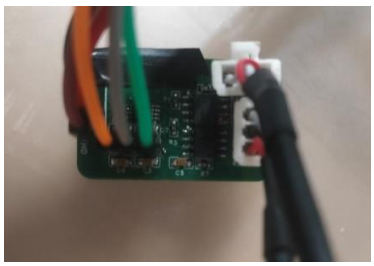


图 4 激光振镜支架

2.2.3 电路各模块介绍



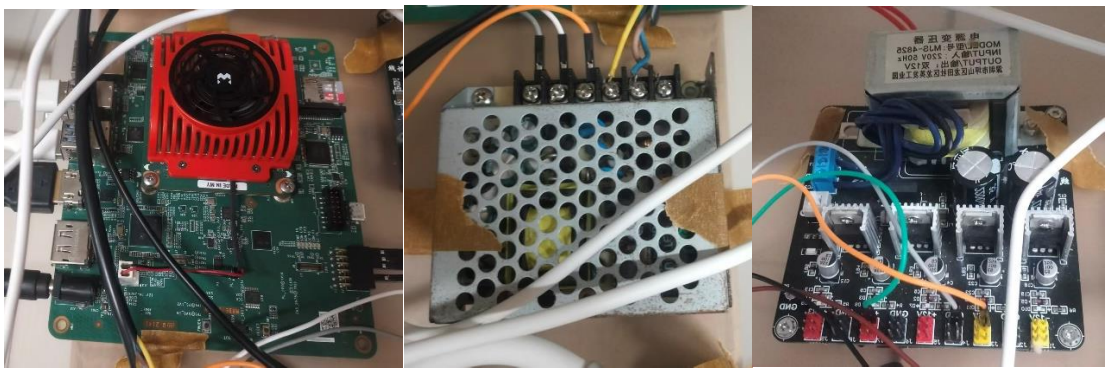
DAC 模块



双目摄像头



激光振镜



KV260　　振镜供电模块　　DAC 供电模块

图 5 电路模块

硬件由双目摄像头、KV260、DAC 模块、激光振镜和电源五部分组成，各模块的主要输入输出如下：

（1）双目摄像头：输入为场景光信号，输出左、右两路图像数据至 KV260，用于垃圾识别、左右目标匹配与稳定确认，并获取目标中心的像素坐标[8]。

（2）KV260 板卡：输入为双目图像数据，板上 DPU 运行量化后的.xmodel，完成目标检测与分类；PS 端执行目标匹配、坐标转换及控制逻辑，通过 AXI 总线调用 PL 端模块，输出 SPI 控制数据和界面显示数据。

（3）DAC 模块：输入为 SPI 数字控制量，输出 X、Y 路模拟电压，用于控制振镜两轴的偏转角度。

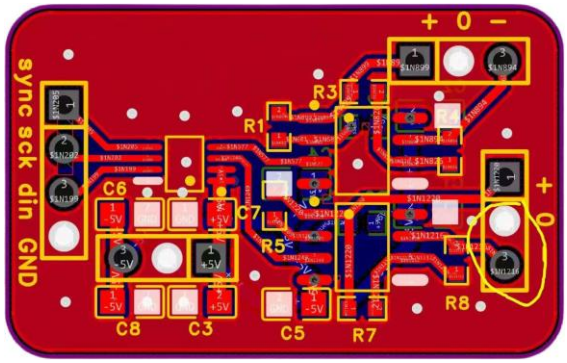


图 6 DAC 模块 PCB 图

（4）激光振镜：系统利用左相机中的目标中心像素坐标，通过标定函数直接计算振镜控制值和目标在标定平面内的实际坐标。控制值经 DAC 转换为 X、Y 模拟电压，驱动振镜调整激光束方向，使光点落到目标位置；实际坐标用于目标位置显示与记录。

（5）电源模块：为振镜驱动器提供 15V 电压，为 DAC 模块提供 5V 供电，保

证系统稳定运行。

2.3 软件系统介绍

2.3.1 软件整体介绍

本系统的软件流程主要包括模型训练、模型适配、量化与编译以及板端部署与运行。首先运行 YOLOv5 工程完成六类垃圾检测模型的训练，然后在 Ubuntu 20.04 虚拟机中准备 Vitis AI 环境，将训练权重转换为 KV260 DPU 架构使用的 .xmodel 文件。各模块通过权重、量化配置和编译结果衔接，整体流程如下[2]。

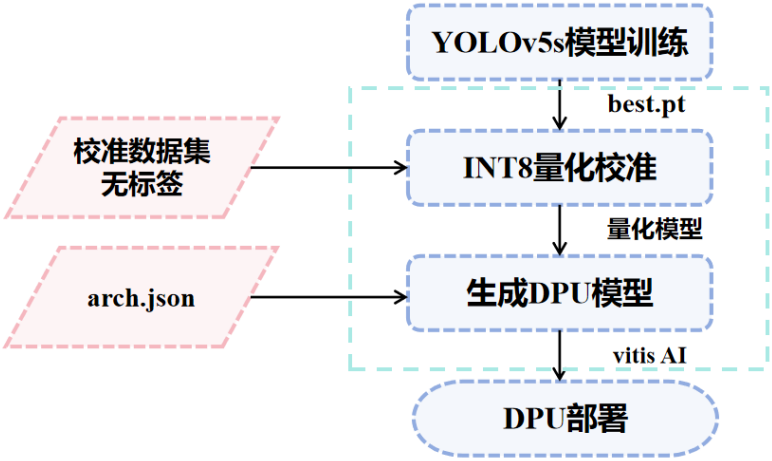


图 7 软件整体流程

2.3.2 软件各模块介绍

（1）模型训练模块

采用 Python 3.8 和 PyTorch，基于 YOLOv5s 网络进行训练。数据集包括金属罐(Cans)、玻璃瓶(Glass_Bottles)、纸类(Paper)、塑料(Plastic)、纺织物(Textile)及其他垃圾(other)六个类别，使用 CPU 完成 150 轮训练。

为适配 KV260 DPU 部署路线，训练前将卷积模块的默认 SiLU 激活改为 ReLU，在 yolov5s.yaml 中添加 activation: nn.ReLU()。程序根据新配置构建网络，并以 yolov5s.pt 初始化可匹配的权重。训练结果保存为 best.pt，作为量化模块的输入。工程训练阶段具体函数流程图和输入输出如下图所示：



图 8 YOLOv5 训练阶段函数流程图

（2）模型适配模块

在 Ubuntu 20.04 中安装 Docker，拉取 xilinx/vitis-ai-cpu:2.5.0 镜像，并启用容器中的 vitis-ai-pytorch 环境。该环境使用 Python 3.7、PyTorch 1.10.1，提供 NNDCT 量化工具和 vai_c_xir 编译器。将训练工程复制到虚拟机，再挂载到容器中运行。

由于 YOLOv5 工程在 Python 3.8 环境下运行，存在 := 等不兼容 Python 3.7 的写法，因此修改代码调整为等价语法，并关闭源码中的自动依赖升级。为解决原工程与量化环境的兼容问题，将加载模型和执行推理所需的网络代码单独整理到量化运行目录中，保留原有权重及网络计算逻辑。

（3）量化与编译模块

编写量化程序 quantize.py ，使用 Vitis AI 提供的 vai_q_pytorch 工具对训练好的模型进行校准与 INT8 量化处理,如图 9 所示。



图 9 量化阶段流程图与输入输出

量化模型导出后，使用 vai_c_xir 和容器提供的 KV260 参考 arch.json 编译，目标架构为 DPUCZDX8G_ISA1_B4096，生成 garbage_yolov5.xmodel，具体函数流程图与输入输出如图 10。

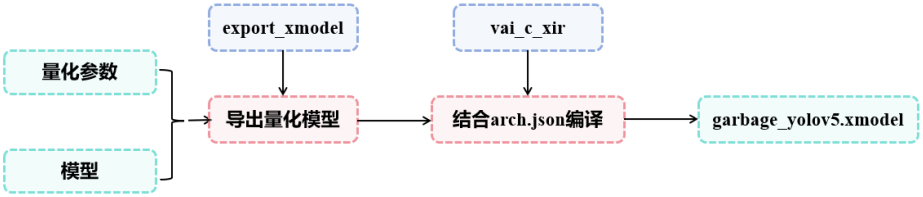


图 10 编译生成.xmodel 文件

（4）模型部署与运行模块

通过 Vitis Linker 构建硬件部署文件.bit、.xclbin，配合设备树及编译后的 .xmodel 模型部署至 KV260。系统在 DDR 中分配输入、输出缓冲区，利用 AXI4 接口完成 PS 与 PL 之间的数据传输。运行过程中，由 VART 调度 DPU 执行模型计算，实现 YOLOv5 的板端推理与目标检测。

第三部分 完成情况及性能参数

3.1 整体介绍（整个系统实物的正面、斜 45° 全局性照片）

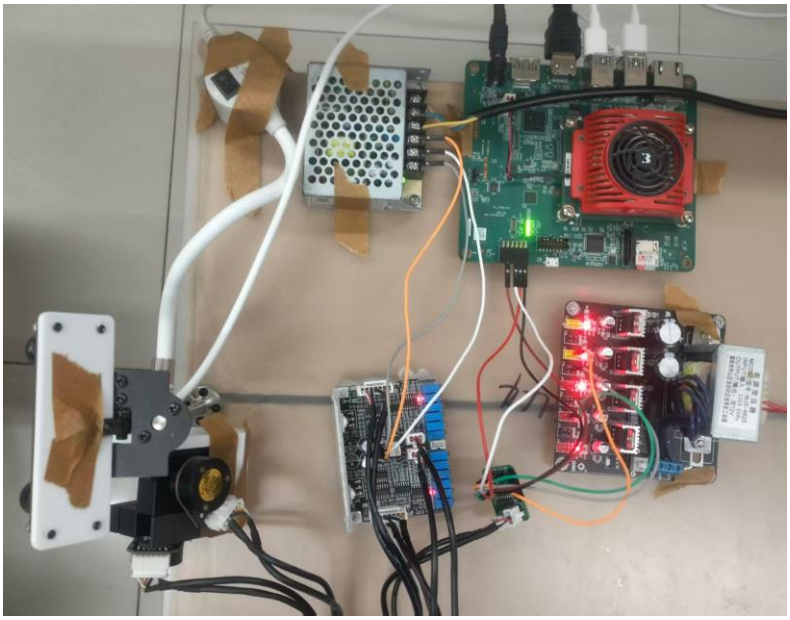


图 11 系统实物正面

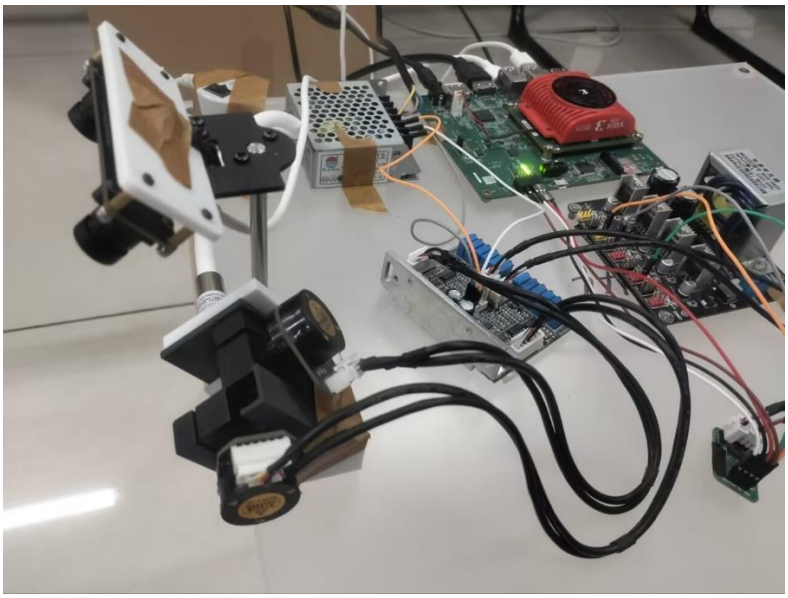


图 12 斜 45° 全局性照片

3.2 工程成果（分硬件实物、软件界面等设计结果）

3.2.1 机械成果

将各模块放置在腿长 35cm 的亚克力置物架上，可保证摄像头观测到地面垃圾，并且激光可以打在目标物中心。同时亚克力材质可有效避免模块短路。

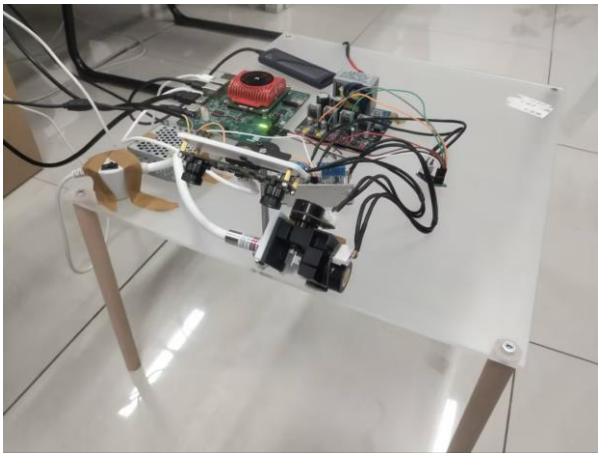


图 13 机械结构实物

3.2.2 电路成果

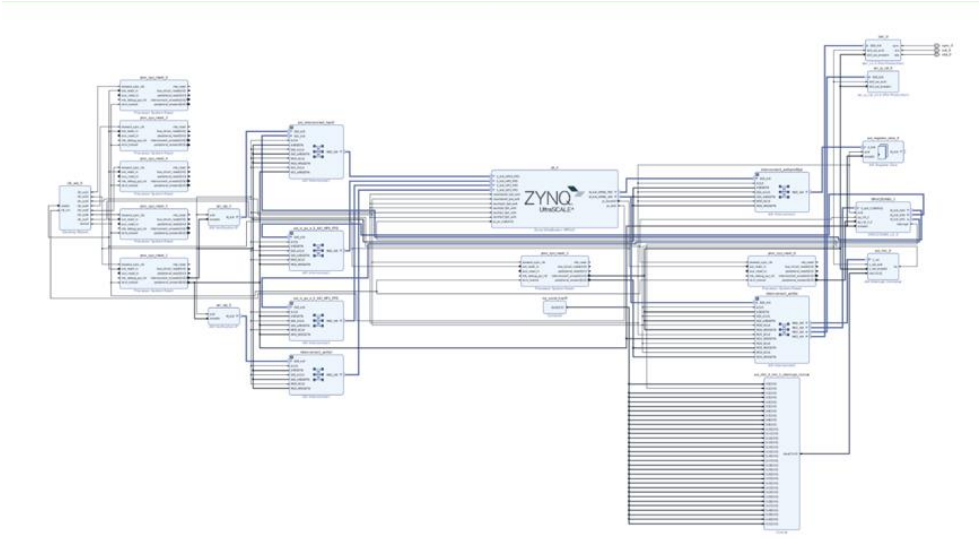


图 14 基于 Zynq 的系统硬件 Block Design 图

3.2.3 软件成果

在板卡上运行程序，打开火狐浏览器，访问 http://127.0.0.1:8765/，模型识别效果良好，程序输出识别对象的种类和坐标。按下开始打扫按键，激光依次标定视野中的垃圾，在同一目标上停留 5s 视为清扫完毕，右侧的进度条显示当

前处理进度。

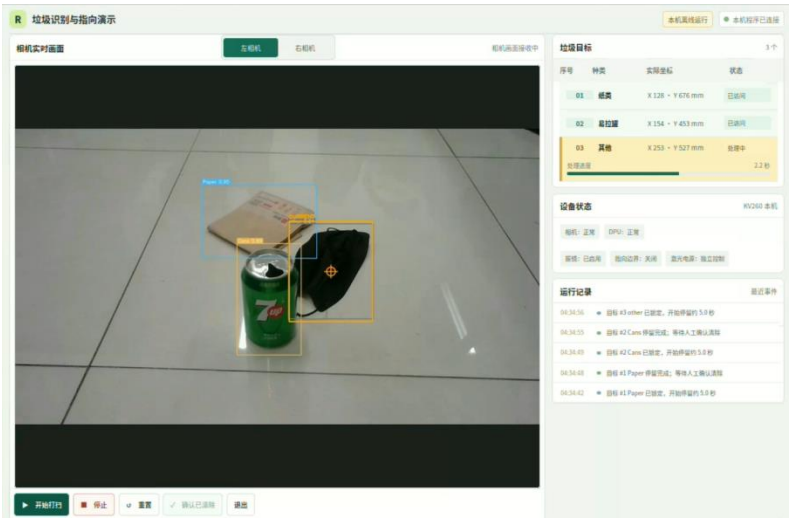


图 15 系统运行界面

3.3 特性成果（逐个展示功能、性能参数等量化指标）

3.3.1 识别准确度

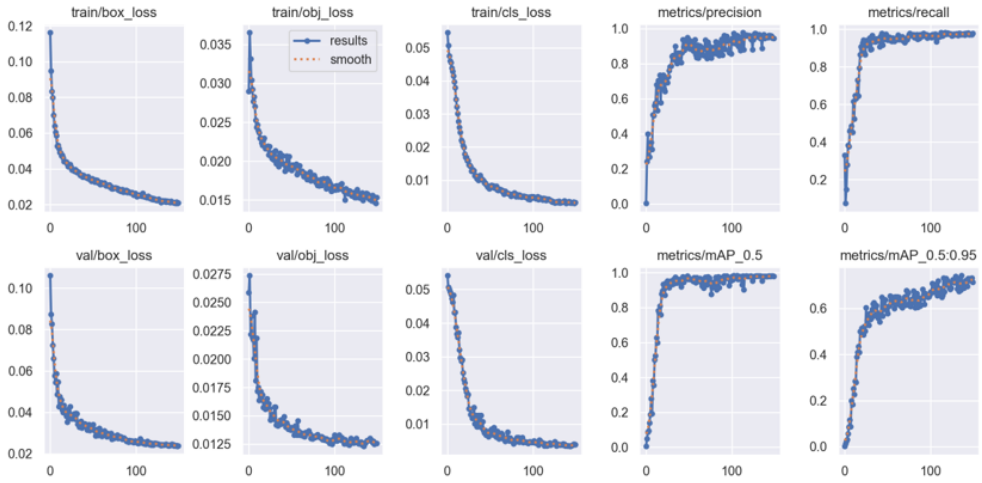


图 16 训练过程曲线

如图，训练与验证损失整体下降，精确率、召回率和 mAP 随训练逐步提高并趋于稳定，表明模型已在当前数据划分下收敛。

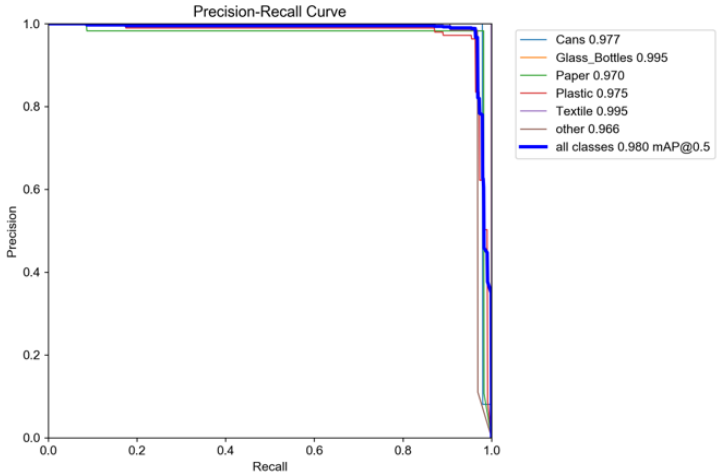


图 17 PR Curve

如图，保存的 PR 曲线显示全类别 mAP@0.5 为 0.980。

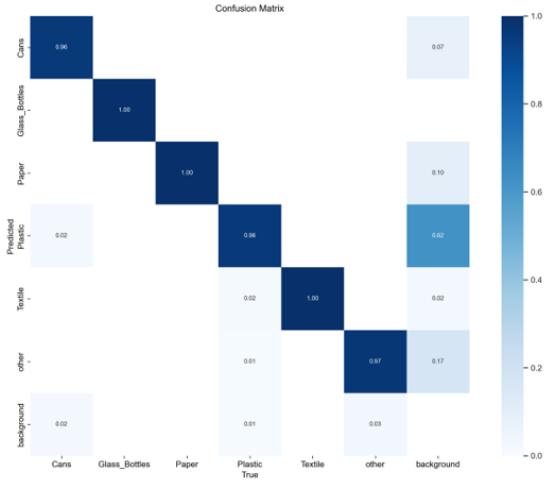
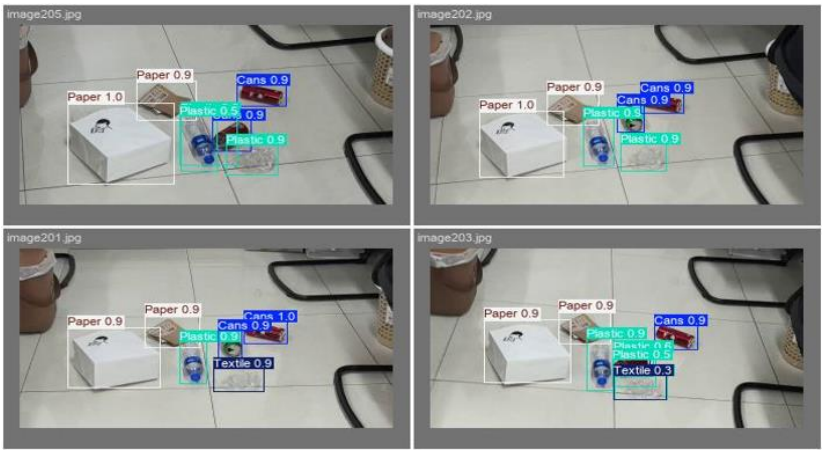


图 18 归一化混淆矩阵

图中的归一化混淆矩阵的对角线数值为 0.96～1.00，类别间混淆较少。背景列中塑料占比为 0.62，表示未匹配预测框中塑料所占比例较高，推测可能因其材质较为透明，容易误识别为地面反光[4]。



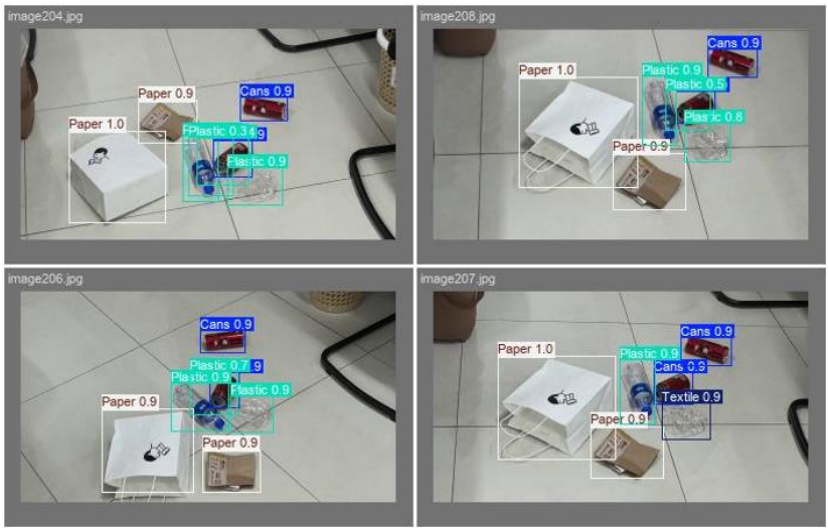


图 19 验证集检测结果示例

3.3.2 推理速度

在输入分辨率为 320×320 的条件下测试，结果表明，DPU 推理平均耗时为 5.96～6.08 ms，仅推理环节的实测帧率为 163.87～167.31 FPS；包含图像预处理、DPU 推理及检测框解码、非极大值抑制等后处理的单目检测流程平均耗时为 43.32～47.75 ms，实测帧率达到 20.92～23.07 FPS，验证了模型在嵌入式平台上的快速检测能力。

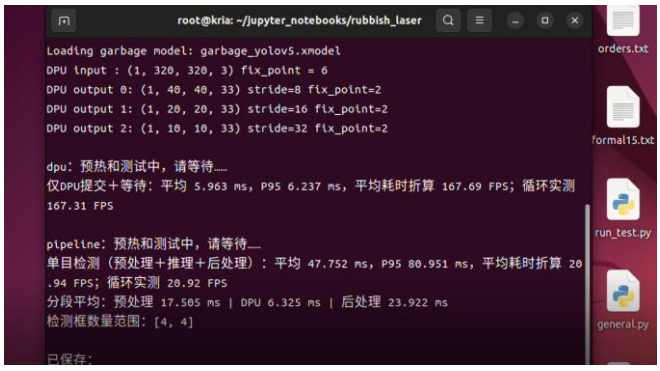


图 20 推理速度测试

3.3.3 定位误差

系统利用左相机中的目标中心像素坐标，通过标定函数计算目标在地面坐标系中的实际位置。采用已有十组标定数据进行二次多项式拟合，X、Y 方向的均方根误差分别约为 4.10 mm、5.18 mm，标定点最大平面误差约为 13.72 mm。

3.3.4 振镜精度

考虑到应用场景较为复杂，且振镜本身存在动态响应延迟，本系统未设计

视觉闭环反馈环节，而是采用开环方式控制光束指向。在高速扫描及角度频繁微调时，振镜的实际偏转角与指令值之间不可避免地存在细小偏差，这一偏差随传播距离增大而被逐渐放大。测算结果表明，当作用距离约为 80 cm 时，系统的总体指向误差处于 2~3 mm 范围内，其中角度微偏差的放大效应是主要误差来源。

第四部分　总结

4.1　可扩展之处

目前系统已打通识别、分类、测距与激光标记的闭环，仍有几个方向可以深入。第一，借助 5G 或 LoRa 实现多台设备协同，在校园、公共道路等较大区域内分布式巡检。第二，在现有硬件上叠加机械抓取、自动清扫和分类投放模块，做成多功能环卫机器人，由 Vitis 统一调度。第三，建立云端平台汇总边缘数据、持续优化模型，并通过 OTA 推送更新，使系统能够自我迭代。此外，多目标跟踪与清扫路径规划也可以纳入后续工作，使方案更加完整。

4.2　心得体会

从组队、联系老师、确定选题到最终提交，备赛前后历时两个月。这是我们第一次参加 FPGA 竞赛，也是第一次完整地以团队方式完成一个项目。

队伍中有成员曾在课程中接触过 YOLOv5s 工程训练，但真正将模型部署到 FPGA 平台后，我们对模型训练、量化、DPU 部署以及硬件协同有了更加深入的认识。随着项目推进，我们逐步完成了 YOLOv5s 训练、模型量化、DPU 部署、双目测距和振镜控制，理解了从算法设计到硬件实现的完整过程。

算法部分让我们体会尤深。受硬件条件限制，模型只能在 CPU 训练，150 轮训练耗时较长，每次调整参数都需要等待很久。因此，我们学会了结合损失曲线和评价指标判断训练方向，再决定是否继续调整。结果出来后，塑料因透明反光，背景误检偏多。这让我们认识到，数据质量和类别均衡会直接影响模型性能。为了适配 DPU，我们还将激活函数由 SiLU 改为 ReLU，也第一次意识到网络结构需要结合实际硬件条件进行调整。

硬件和部署环节同样不轻松。DPU 无法生成、Vivado 综合报错、量化流程不顺利等问题反复出现，往往需要查阅文档、修改配置并重新测试。完成模型部署后，我们还对置信度、检测数量、振镜范围和坐标偏移进行了多次调节，并通

过测距实验和多点标定，逐步建立像素位置、实际地面坐标与振镜 X、Y 指令之间的对应关系。双目测距和振镜指向也需要结合实际装置反复验证，不能只依赖理论计算。

几个人围着屏幕讨论方案、反复调试程序、晚上留下来排查问题的情形，至今记忆犹新。在反复调参和完善方案的过程中，我体会到算法效果、硬件参数和软件流程之间相的互影响，只有不断测试和修正，才能让系统逐渐稳定下来。

在此要感谢指导老师一路以来的耐心指导，也感谢队友之间的相互支持与鼓励。这段经历让我们明白，许多问题不可能一次解决，但只要坚持排查、持续推进，就总能找到解决问题的方法。

第五部分 参考文献

[1] AMD. (2022). Kria KV260 Vision AI starter kit user guide (UG1089) [Technical documentation]. Retrieved October 24, 2026, from https://docs.amd.com

[2] AMD. (2023). Vitis AI user guide (UG1414) [Technical documentation]. Retrieved October 24, 2026, from https://docs.amd.com

[3] Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection (arXiv:2004.10934). arXiv.https://doi.org/10.48550/arXiv.2004.10934

[4] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In D. Fleet, T. Pajdla, B. Schiele, & T. Tuytelaars (Eds.), Computer vision – ECCV 2014 (Lecture Notes in Computer Science, Vol. 8693, pp. 740–755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48

[5] Proença, P. F., & Simões, P. (2020). TACO: Trash annotations in context for litter detection (arXiv:2003.06975). arXiv. https://doi.org/10.48550/arXiv.2003.06975

[6] Qiu, J., Wang, J., Yao, S., Guo, K., Li, B., Zhou, E., Yu, J., Tang, T., Xu, N., Song, S., Wang, Y., & Yang, H. (2016). Going deeper with embedded FPGA platform for convolutional neural network. In Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (pp.

26–35). Association for Computing Machinery. https://doi.org/10.1145/2847263.2847265

[7] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (pp. 779–788). IEEE. https://doi.org/10.1109/CVPR.2016.91

[8] Scharstein, D., & Szeliski, R. (2002). A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. International Journal of Computer Vision, 47(1–3), 7–42. https://doi.org/10.1023/A:1014573219977